



Estatística Computacional com R (PPGBC/UFPA e PPGCTIF/UFOPA)



Introdução ao R (Estrutura de Dados no R)



Prof. Dr. Deivison Venicio Souza

Universidade Federal do Pará (UFPA)

Faculdade de Engenharia Florestal

Laboratório de Manejo Florestal, Tecnologias e Comunidades Amazônicas

E-mail: deivisonvs@ufpa.br

1ª versão: 14/outubro/2022
(Atualizado em: 28/junho/2026)



PPGCTIF



Objetivos



Ao final desta aula, espera-se que os discentes sejam capazes de:

- Compreender o papel das estruturas de dados na linguagem R;
- Criar e manipular vetores, matrizes, arrays, listas e data frames;
- Reconhecer as características e aplicações de cada estrutura;
- Utilizar funções básicas para construção de estruturas de dados; e
- Relacionar estruturas de dados a problemas práticos de análise de dados.



Conteúdo



Parte 4 – Estrutura de dados

1. Vetores

- Classes
- Criação de vetores
- `c()`, `seq()`, `rep()` e `scan()`
- Factors e níveis

2. Matrizes

- Propriedades
- Função `matrix()`
- Nomeando dimensões
- Imagens em escala de cinza



Conteúdo



Parte 4 – Estrutura de dados

3. Arrays

- Propriedades
- Função `array()`
- Nomeando dimensões
- Imagens RGB

4. Listas

- Propriedades
- Função `list()`

5. Data frames

- Propriedades
- Função `data.frame()`



Parte 4

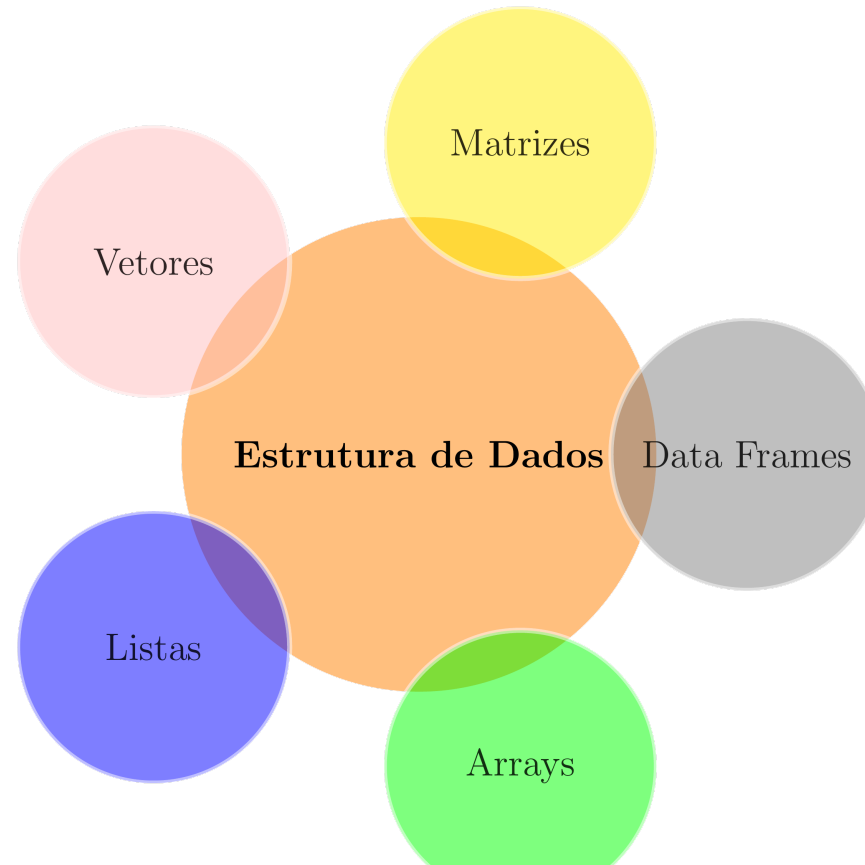
Estrutura de Dados em R

Vetores



Estrutura de dados em R

- O R pode armazenar dados em diferentes estruturas, cada uma adequada para uma finalidade específica.



Comparando as estruturas de dados



Estrutura	Dimensão	Tipo de dados	Exemplo
Vetor	1D	Homogêneo	DAP de árvores
Matriz	2D	Homogêneo	Imagem em escala de cinza
Array	3D ou mais	Homogêneo	Imagem RGB
Data Frame	2D	Heterogêneo	Inventário florestal
Lista	Flexível	Heterogêneo	Resultado de análises

💡 Em R, cada estrutura possui uma forma específica de armazenar e organizar os dados.



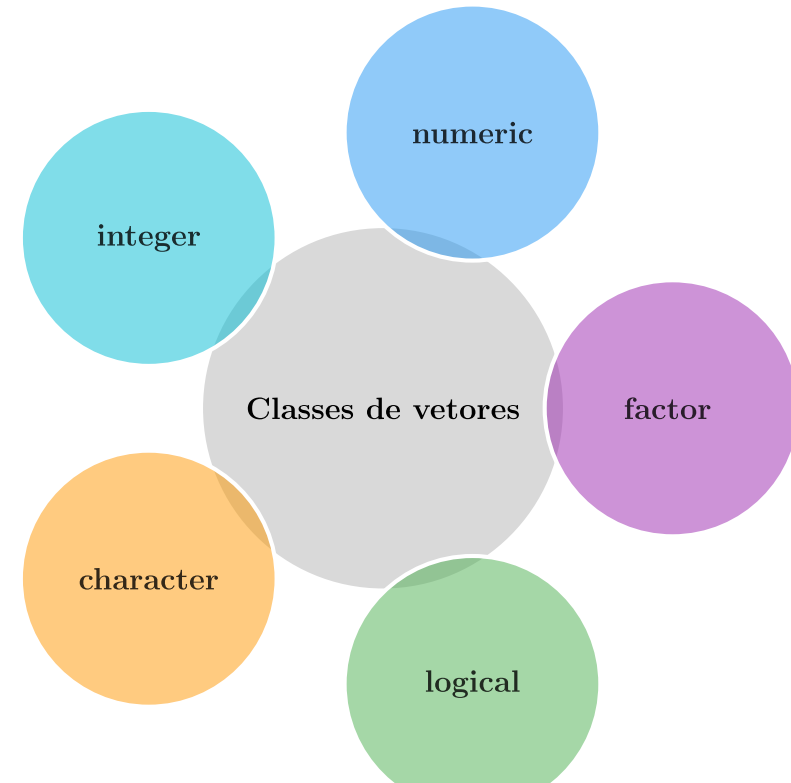
Estrutura de dados em R



Vetores

- São a estrutura de dados mais básica do R;
- Possuem apenas uma dimensão (1D);
- Todos os elementos devem pertencer à mesma classe;
- Servem de base para a construção de estruturas mais complexas.

Principais classes de vetores



Estrutura de dados em R



Classes de Vetores

Classe	Tipo de armazenamento	Exemplo	Aplicação
integer	integer	<code>c(1L, 2L, 3L)</code>	Quantidade de árvores
numeric	double	<code>c(12.5, 15.8, 20.1)</code>	DAP, altura, volume
character	character	<code>c("Ipê", "Mogno")</code>	Nome das espécies
logical	logical	<code>c(TRUE, FALSE)</code>	Sim/Não
factor	integer	<code>factor(c("Alta", "Baixa"))</code>	Classes ou categorias

💡 Além da classe do objeto, é importante compreender o tipo de armazenamento dos dados.



Integer × Double

Integer (Números inteiros)

```
x <- c(1L, 2L, 3L)

class(x)      # Classe
typeof(x)     # Tipo interno
```

```
[1] "integer"
[1] "integer"
```

Double (Números reais)

```
y <- c(12.5, 15.8, 20.1)

class(y)      # Classe
typeof(y)     # Tipo interno
```

```
[1] "numeric"
[1] "double"
```



Em R, números são armazenados como **double** por padrão. Use **L** para criar valores **integer**.

Factor: Como o R armazena categorias?



```
especie <- factor(c("Mogno",  
                    "Teca",  
                    "Ipê"))
```

```
class(especie)  
typeof(especie)
```

```
[1] "factor"  
[1] "integer"
```

O usuário vê:

Mogno
Teca
Ipê

O R armazena:

2
3
1

💡 Fatores representam categorias, mas são armazenados internamente como números inteiros.

💡 Por padrão, o R atribui códigos inteiros aos níveis do fator em ordem alfabética.



Estrutura de dados em R



Como criar vetores?

Existem diversas formas de criar vetores na linguagem R.

Mais utilizadas

- `c()` (*concatenação*)
- `seq()` (*sequência*)
- `rep()` (*replicação*)

Entrada manual

- `scan()`



A função `c()` é a forma mais utilizada de criar vetores no R.



Estrutura de dados em R



Vetores - Função c()

Classe "character"

```
especie <- c("Acapu",  
             "Mogno",  
             "Cedro",  
             "Ipê")
```

```
class(especie)
```

```
[1] "character"
```

Classe "factor"

```
especie <- factor(c("Mogno",  
                    "Teca",  
                    "Ipê",  
                    "Mogno"))
```

```
class(especie)
```

```
[1] "factor"
```



Factor informa ao R que os valores representam **categorias**, e não textos livres (**character**).



Estrutura de dados em R



Vetores - Função c()

Classe "integer"

```
idade <- c(5L,  
          8L,  
          12L,  
          15L)
```

```
class(idade)
```

```
[1] "integer"
```

Classe "numeric"

```
diametro <- c(23.0,  
              27.0,  
              33.6,  
              42.6)
```

```
class(diametro)
```

```
[1] "numeric"
```



Vetores **integer** armazenam números inteiros. Vetores **numeric** armazenam números reais.



Estrutura de dados em R



Vetores - Classe "logical"

```
cipo <- c(TRUE,  
          FALSE,  
          FALSE,  
          TRUE)
```

```
class(cipo)
```

```
[1] "logical"
```

Exemplo:

TRUE → Presença de cipó

FALSE → Ausência de cipó



Estrutura de dados em R



Factor: níveis não ordenados

Em vetores da classe **factor**, os níveis (*levels*) são, por padrão, organizados em **ordem alfabética**.

👉 Níveis de risco de queda de árvores

```
risco <- factor(  
  x = c("Critico",  
        "Baixo",  
        "Moderado")  
)  
  
levels(risco)
```

```
## [1] "Baixo" "Critico" "Moderado"
```



A ordem alfabética nem sempre representa a hierarquia real das categorias.



Estrutura de dados em R



Factor: níveis ordenados

A **ordem dos níveis** pode ser definida usando o argumento `levels`.

Hierarquia desejada:

Baixo < Moderado < Critico

👉 **Níveis de risco de queda de árvores**

```
risco <- factor(  
  x = risco,  
  levels = c("Baixo",  
             "Moderado",  
             "Critico"),  
  ordered = TRUE  
)  
  
levels(risco)
```

```
## [1] "Baixo" "Moderado" "Critico"
```



Estrutura de dados em R



Vetores - Função seq() (sequence)

`seq(from = ?, to = ?, by = ?, length.out = ?, along.with = ?)`

from = valor inicial da sequência.

to = valor final da sequência.

by = valor incremental da sequência.

length.out = quantidade de elementos da sequência.

along.with = cria uma sequência com o mesmo comprimento de um vetor.



Estrutura de dados em R



Vetores - Função seq() (sequence)

```
# Sequência de 1 a 10  
seq(10)
```

```
# Sequência de 1 a 10  
seq(1:10)
```

```
# Sequência de 1 a 10, com incremento 1  
seq(from = 1, to = 10, by = 1)
```

```
# Sequência de -2 a 10, com incremento 2  
seq(from = -2, to = 10, by = 2)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
## [1] -2 0 2 4 6 8 10
```

💡 O argumento `by` controla o incremento entre os elementos da sequência.



Estrutura de dados em R



Vetores - Função seq() (sequence)

```
# Sequência de 5 números entre 0 e 1
seq(from = 0, to = 1, length.out = 5)

# Sequência do tamanho do vetor
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")

seq(along.with = especie)
```

```
## [1] 0.00 0.25 0.50 0.75 1.00
```

```
## [1] 1 2 3 4
```

💡 Use `length.out` para definir a quantidade de elementos e `along.with` para gerar uma sequência do tamanho do vetor.



Estrutura de dados em R



Vetores - Função rep() (replicate)

```
rep(x, times = 1, length.out = NA, each = 1)
```

x = O objeto (vetor, lista, número) que você deseja repetir.

times = Quantidade de repetições da sequência inteira.

each = Quantidade de repetições de cada elemento individual em sequência.

length.out = Define o tamanho total final desejado para o vetor.



Estrutura de dados em R



Vetores - Função rep() - Argumento: times

```
# Repete o vetor 2 vezes  
rep(x = c("Mogno", "Ipê"), times = 2)  
  
# Repete o vetor 3 vezes  
rep(x = c("Mogno", "Ipê"), times = 3)
```

```
## [1] "Mogno" "Ipê" "Mogno" "Ipê"
```

```
## [1] "Mogno" "Ipê" "Mogno" "Ipê" "Mogno"
```



O argumento times repete o vetor inteiro.



Estrutura de dados em R



Vetores - Função rep() - Argumentos: each e times

```
# Repete cada espécie 2x
rep(x = c("Mogno", "Ipê"), each = 2)

# Repete cada espécie 2x, vetor 3x
rep(x = c("Mogno", "Ipê"), each = 2, times = 3)
```

```
## [1] "Mogno" "Mogno" "Ipê"  "Ipê"
```

```
## [1] "Mogno" "Mogno" "Ipê"  "Ipê"  "Mogno" "Mogno" "Ipê"  "Ipê"  "Mogno"
## [10] "Mogno" "Ipê"  "Ipê"
```

💡 O argumento `each` repete cada elemento, enquanto `times` repete o vetor.



Estrutura de dados em R



Vetores - Função rep() - Argumento: length.out

```
# Repete até obter 3 elementos  
rep(x = c("Mogno", "Ipê"), length.out = 3)
```

```
# Repete até obter 5 elementos  
rep(x = 1:2, length.out = 5)
```

```
## [1] "Mogno" "Ipê" "Mogno"
```

```
## [1] 1 2 1 2 1
```



O argumento `length.out` define o comprimento final do vetor.



Estrutura de dados em R



Vetores - Função scan()

A função `scan()` permite inserir dados manualmente pelo R Console.

```
y <- scan()
```

Procedimento:

1. Digite o comando `scan()`.
2. Digite um valor por linha e pressione **ENTER**.
3. Para finalizar, pressione **ENTER** duas vezes.

💡 A função `scan()` cria vetores a partir de dados digitados pelo usuário.



Estrutura de dados em R



Vetores - Função scan()

Atividade para 🏠

Utilize a função `scan()` para criar vetores para as variáveis da tabela abaixo:

```
##      especie diametro altura
## 1      Acapu      23.0     8.5
## 2 Araucaria      27.0     9.2
## 3      Mogno      33.6    10.5
```



Para criar vetores da classe **character**, utilize:

```
scan(what = character())
```



Parte 4

Estrutura de Dados em R

Matrizes



Estrutura de dados em R



Matrizes - Propriedades

- São uma extensão dos vetores para duas dimensões (linhas e colunas).
- Possuem duas dimensões: linhas e colunas.
- Todos os elementos devem pertencer à mesma classe.
- Podem ser criadas com `matrix()`, `cbind()` ou `rbind()`.
- A função `matrix()` é a forma mais utilizada.

```
matrix( data = NA, nrow = 1, ncol = 1, byrow = FALSE,  
        dimnames = NULL )
```

data = vetor de dados.

nrow = número de linhas.

ncol = número de colunas.

byrow = preenchimento por linhas (TRUE) ou colunas (FALSE).

dimnames = nomes das linhas e colunas.



Estrutura de dados em R



Matrizes - Função matrix()

```
# Cria uma matriz 2 x 3
matrix(
  data = c(23, 27, 31, 25, 29, 34),
  nrow = 2,
  ncol = 3
)
```

```
##      [,1] [,2] [,3]
## [1,]  23  31  29
## [2,]  27  25  34
```

💡 Por padrão, a função `matrix()` preenche a matriz por colunas.



Estrutura de dados em R



Matrizes - Argumento: dimnames

```
# Cria uma matriz 2 x 3, nomeada
matrix(
  data = c(23, 27, 31, 25, 29, 34),
  nrow = 2,
  ncol = 3,
  dimnames = list(
    c("Arv1", "Arv2"),
    c("Parc1", "Parc2", "Parc3")
  )
)
```

##		Parc1	Parc2	Parc3
##	Arv1	23	31	29
##	Arv2	27	25	34

💡 O argumento `dimnames` permite atribuir nomes às linhas e colunas da matriz.



Estrutura de dados em R



Matrizes - Imagem digital - (escala de cinza)



Imagem original (1024 × 1024 pixels)

```
library(png)
mat <- readPNG("fig/leaf_gray.png")
dim(mat)
View(mat)
```

Matriz de pixels



Parte 4

Estrutura de Dados em R

Arrays



Estrutura de dados em R



Arrays - Propriedades

- São uma generalização das matrizes.
- **Matrizes** possuem duas dimensões (linhas e colunas).
- **Arrays** podem possuir duas ou mais dimensões.
- Todos os elementos devem pertencer à mesma classe.
- Podem ser criados com a função `array()`.

💡 Vetores, matrizes e arrays diferem principalmente pelo número de dimensões.

```
array( data = NA, dim = length(data), dimnames = NULL )
```

data = vetor de dados.

dim = vetor que define as dimensões do array.

dimnames = nomes das dimensões.



Estrutura de dados em R



Arrays - Função array()

```
# Cria um array 3 x 2 x 2
array(
  data = 1:12,
  dim = c(3, 2, 2)
)
```

💡 Por padrão, os elementos são distribuídos por colunas em cada camada do array.

```
## , , 1
##
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
##
## , , 2
##
##      [,1] [,2]
## [1,]    7   10
## [2,]    8   11
## [3,]    9   12
```



Estrutura de dados em R

Arrays - Função array() - Nomeando dimensões

```
array(  
  data = 1:18,  
  dim = c(3, 3, 2),  
  dimnames = list(  
    c("L1", "L2", "L3"),  
    c("C1", "C2", "C3"),  
    c("M1", "M2")  
  )  
)
```

💡 O argumento `dimnames` permite nomear as dimensões do array.

```
## , , M1  
##  
##      C1 C2 C3  
## L1   1  4  7  
## L2   2  5  8  
## L3   3  6  9  
##  
## , , M2  
##  
##      C1 C2 C3  
## L1  10 13 16  
## L2  11 14 17  
## L3  12 15 18
```

Estrutura de dados em R



Arrays - Imagem digital - RGB



Imagem colorida (RGB)

```
library(png)

arr <- readPNG("fig/leaf.png")

dim(arr)

View(arr)
```

💡 Imagens coloridas são armazenadas como arrays: linhas $\times$ colunas $\times$ canais RGB.



Parte 4

Estrutura de Dados em R

Data Frames



Estrutura de dados em R



Data Frames - Propriedades

- São estruturas **bidimensionais** organizadas em linhas e colunas.
- Permitem combinar vetores de diferentes classes, desde que possuam o mesmo comprimento.
- O R-base possui a função `data.frame()`.
- Seus elementos podem ser acessados e modificados por indexação.
- Em geral, tabelas importadas de arquivos (.csv, .txt e .xlsx) são armazenadas como data frames.

Atividade para 🏠

Use o comando `x <- edit(data.frame())` e crie a tabela a seguir:

##	especie	diametro	altura
## 1	Acapu	23.0	8.5
## 2	Araucaria	27.0	9.2
## 3	Mogno	33.6	10.5
## 4	Cedro	42.6	13.4
## 5	Ipe	52.1	15.8



Estrutura de dados em R

Data Frames - Função data.frame()

Cria um data frame a partir de vetores existentes

```
especie <- c("Acapu", "Araucaria", "Cedro", "Tauari")
diametro <- c(23.0, 27.0, 33.6, 52.6)
altura <- c(8.4, 8.7, 9.1, 18.2)
cortar <- c(FALSE, FALSE, FALSE, TRUE)

data.frame(especie, diametro, altura, cortar)
```

Resultado

##	especie	diametro	altura	cortar
## 1	Acapu	23.0	8.4	FALSE
## 2	Araucaria	27.0	8.7	FALSE
## 3	Cedro	33.6	9.1	FALSE
## 4	Tauari	52.6	18.2	TRUE



Um data frame pode ser criado pela combinação de vetores com o mesmo comprimento.

Estrutura de dados em R



Data Frames - Função data.frame()

Usando diretamente a função data.frame()

```
(invFlor <-  
  data.frame(  
    especie = c("Acapu", "Araucaria", "Cedro", "Taua"  
    diametro = c(23.0, 27.0, 33.6, 52.6),  
    altura = c(8.4, 8.7, 9.1, 18.2),  
    cortar = c(FALSE, FALSE, FALSE, TRUE)  
  ))
```

Resultado

##	especie	diametro	altura	cortar
## 1	Acapu	23.0	8.4	FALSE
## 2	Araucaria	27.0	8.7	FALSE
## 3	Cedro	33.6	9.1	FALSE
## 4	Tauari	52.6	18.2	TRUE



A função data.frame() permite criar tabelas diretamente a partir dos valores das variáveis.



Estrutura de dados em R



Data Frames - Explorando a estrutura

Explorando um data frame

```
# Número de linhas e colunas  
dim(invFlor)  
  
# Estrutura do objeto  
str(invFlor)
```

Resultado

```
## [1] 4 4  
  
## 'data.frame': 4 obs. of 4 variables:  
## $ especie : chr "Acapu" "Araucaria" "Cedro" "Ta  
## $ diametro: num 23 27 33.6 52.6  
## $ altura : num 8.4 8.7 9.1 18.2  
## $ cortar : logi FALSE FALSE FALSE TRUE
```



A função `str()` permite visualizar a estrutura e as classes das colunas de um data frame.



Parte 4

Estrutura de Dados em R

Listas



Estrutura de dados em R



Listas - Propriedades

- Pode reunir diferentes estruturas de dados (vetores, matrizes, data frames e inclusive outras listas).
- O R-base possui a função `list()`.
- Se os objetos já existem pode-se simplesmente adicioná-los à função `list()`.



Estrutura de dados em R



Listas - Função list()

Uma lista a partir de objetos já existentes

```
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
diametro <- c(23.0, 27.0, 33.6, 52.6)
altura <- c(8.4, 8.7, 9.1, 18.2)
```

```
# Sem atribuir nomes aos objetos da lista
(list1 <- list(especie, diametro, altura))
```

```
# Atribuindo nomes aos objetos da lista
(list2 <- list(Esp = especie,
              DAP = diametro,
              H = altura))
```

Nomear os componentes da lista facilita a identificação e a indexação.

```
## [[1]]
## [1] "Acapu" "Mogno" "Cedro" "Ipe"
##
## [[2]]
## [1] 23.0 27.0 33.6 52.6
##
## [[3]]
## [1] 8.4 8.7 9.1 18.2
```

```
## $Esp
## [1] "Acapu" "Mogno" "Cedro" "Ipe"
##
## $DAP
## [1] 23.0 27.0 33.6 52.6
##
## $H
## [1] 8.4 8.7 9.1 18.2
```



Estrutura de dados em R



Listas - Função list()

Uma lista com diferentes estruturas de dados

```
diametro <- c(23.0, 27.0, 33.6, 52.6)

mat <-
  matrix(data=1:6, nrow=2,
         ncol=3, byrow=TRUE,
         dimnames=list(c("L1", "L2"),
                       c("C1", "C2", "C3")))

invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Ipe"),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE)

# Criando lista nomeada
(list3 <- list(Vetor = diametro,
              Matriz = mat,
              DataFrame = invFlor))
```

```
## $Vetor
## [1] 23.0 27.0 33.6 52.6
##
## $Matriz
##      C1 C2 C3
## L1    1  2  3
## L2    4  5  6
##
## $DataFrame
##      especie cortar
## 1    Acapu     Não
## 2    Mogno     Não
## 3    Cedro     Não
## 4      Ipe      Sim
```

Explore as funções `length()` e `names()`.



Estrutura de dados em R



Listas - Coagindo lista para data frame

- Uma lista pode ser transformada em um data frame usando a função `as.data.frame()`.
- Mas, para isso a lista a ser transformada deve possuir vetores de igual comprimento.

Lista para Data Frame

```
# Criando lista nomeada
list4 <- list(
  Esp = c("Acapu", "Mogno", "Cedro", "Ipe"),
  DAP = c(23.0, 27.0, 33.6, 52.6),
  H = c(8.4, 8.7, 9.1, 18.2))

as.data.frame(list4)
```

```
##      Esp  DAP    H
## 1 Acapu 23.0  8.4
## 2 Mogno 27.0  8.7
## 3 Cedro 33.6  9.1
## 4  Ipe 52.6 18.2
```



Dúvidas?



Mensagem

Entender como os dados são armazenados em R é fundamental para aprender a acessá-los, organizá-los e manipulá-los de forma eficiente.



Próximo passo

Aprender técnicas de indexação e operadores para acessar e manipular dados em R.



Perguntas, comentários ou sugestões?

Obrigado!



@lmftca_ufpa



<https://www.lmftca.com.br/>

